

Bài tập trên lớp – Môn Lập trình mạng

Họ và tên: Nguyễn Thu Trang

MSSV: 20235441

Bài 1:

- *Đề bài:*

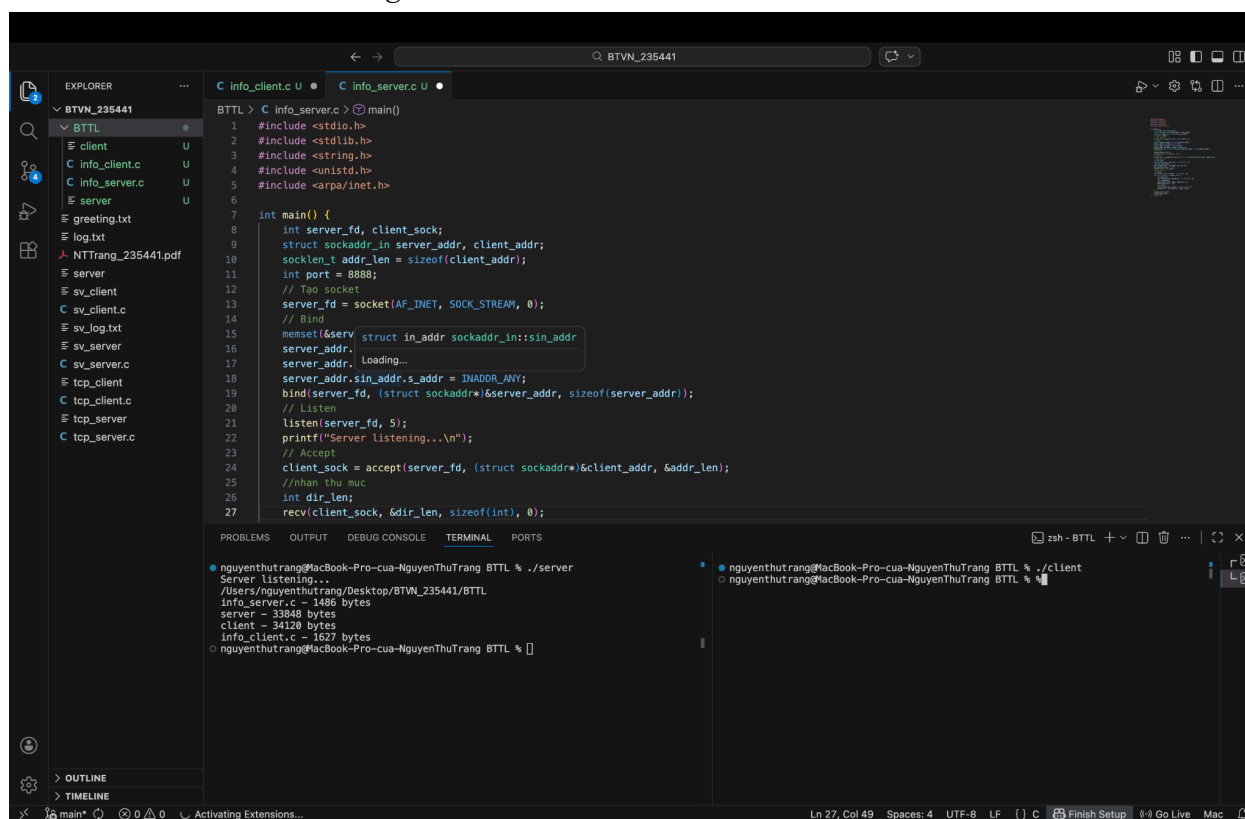
- Ứng dụng info_client lấy tên thư mục hiện tại, danh sách các tập tin và kích thước (trên máy client). Các dữ liệu này sau đó được đóng gói và chuyển sang info_server.
- Ứng dụng info_server nhận dữ liệu từ info_client, tách các dữ liệu và in ra màn hình.
- Yêu cầu kích thước dữ liệu được gửi là nhỏ nhất (không truyền các dữ liệu dư thừa).
- *Ví dụ:*

/mnt/c/Users/admin/source/baitap

bai1.c – 12871 bytes

bai2.c – 1437 bytes

- *Hình ảnh demo chương trình:*



Giải thích:

Phía server: Chạy chương trình info_server.

Phía client: Chạy chương trình info_client.

- Server khởi động thành công và ở trạng thái chờ kết nối. Client kết nối và gửi yêu cầu đến server.
- Server nhận dữ liệu, xử lý (đọc thư mục). Server hiển thị danh sách file và kích thước tương ứng.

- Chương trình hoạt động đúng theo mô hình client-server.

Bài 2:

- *Đề bài:*
 - Ứng dụng client liên tục gửi dữ liệu văn bản sang server.
 - Ứng dụng server nhận dữ liệu từ client, cập nhật và in ra màn hình số lần xuất hiện chuỗi ký tự “0123456789” với dữ liệu đã nhận được.
 - Cần xử lý trường hợp khi chuỗi “0123456789” nằm giữa 2 lần truyền.

Ví dụ dữ liệu gửi từ client sau 4 lần (không bao gồm ký tự xuống dòng):

**SOICTSOICT012345678901234567890123456789012345
6789SOICTSOICTSOICT012345678901234567890123456
7890123456789012345678901234567890123456789012
3456789SOICTSOICT01234567890123456789012345678**

- *Hình ảnh demo chương trình:*

```

C:\info_client.c C:\info_server.c C:\str_server.c U C:\str_client.c U
BTTL > C:\str_server.c > ...
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <sys/socket.h>
5 #include <arpa/inet.h>
6 #include <unistd.h>
7
8 #define PORT 9000
9 #define BUFFER_SIZE 1024
10 #define TARGET "0123456789"
11 #define TARGET_LEN 10
12
13 int count_occurrences(const char *str, const char *target) {
14     int count = 0;
15     const char *tmp = str;
16     while ((tmp = strstr(tmp, target))) {
17         count++;
18         tmp += 1;
19     }
20     return count;
21 }
22
BTTL > C:\str_client.c > main(int, char *[])
8 #define BUFFER_SIZE 1024
9
10 int main(int argc, char *argv[]) {
11     if (argc != 3) {
12         printf("Sử dụng: %s <IP Server> <Cổng>\n", argv[0]);
13         return 1;
14     }
15     int client_sock = socket(AF_INET, SOCK_STREAM, 0);
16     struct sockaddr_in server_addr;
17     server_addr.sin_family = AF_INET;
18     server_addr.sin_port = htons(atoi(argv[2]));
19     inet_pton(AF_INET, argv[1], &server_addr.sin_addr);
20     if (connect(client_sock, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
21         perror("Kết nối thất bại");
22         return 1;
23     }
24     printf("Đã kết nối. Nhập chuỗi để gửi (Ctrl+C để thoát):\n");
25     char buffer[BUFFER_SIZE];
26     while (1) {
27         printf("> ");
28         if (fgets(buffer, sizeof(buffer), stdin) == NULL) break;
29         buffer[strlen(buffer)] = 0;
30         send(client_sock, buffer, strlen(buffer), 0);
31     }
32     return 0;
33 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang BTTL % ./str_server
Server streaming đang chờ kết nối tại port 9000...
Nhan 46 bytes. So lan xuất hiện mỗi: 3. Tổng cộng: 3
Nhan 46 bytes. So lan xuất hiện mỗi: 3. Tổng cộng: 6
Nhan 46 bytes. So lan xuất hiện mỗi: 5. Tổng cộng: 11
Nhan 46 bytes. So lan xuất hiện mỗi: 3. Tổng cộng: 14
nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang BTTL %

```

```

nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang BTTL % ./str_client
Sử dụng: ./str_client <IP Server> <Cổng>
nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang BTTL % ./str_client 127.0.0.1 9000
Đã kết nối. Nhập chuỗi để gửi (Ctrl+C để thoát):
> SOICTSOICT012345678901234567890123456789012345
> 6789SOICTSOICTSOICT012345678901234567890123456
> 7890123456789012345678901234567890123456789012
> 3456789SOICTSOICT01234567890123456789012345678
> ^C
nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang BTTL %

```

Giải thích:

Phía client: Chạy chương trình str_client.c

Phía server: Chạy chương trình str_server.c

- Client gửi dữ liệu văn bản nhiều lần đến server (dạng streaming).
- Server nhận từng phần dữ liệu, xử lý và đếm số lần xuất hiện chuỗi “012345689”.
- Server cập nhật số lần xuất hiện mới và tổng số lần sau mỗi lần nhận.
- Chương trình xử lý đúng trường hợp chuỗi bị chia giữa các lần gửi.

Bài 3:

- *Đề bài:*
 - Ứng dụng udp_echo nhận dữ liệu và phản hồi lại chính dữ liệu đó cho bên truyền.

- *Hình ảnh demo chương trình:*

The screenshot shows a code editor with two files open: `udp_echo_server.c` and `udp_echo_client.c`. Below the code, there are two terminal windows showing the execution of these programs.

udp_echo_server.c:

```

1  int main(int argc, char *argv[]) {
2      if (argc != 2) {
3          printf("Sử dụng: %s <Port>\n", argv[0]);
4          return 1;
5      }
6      int server_sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
7      struct sockaddr_in server_addr;
8      memset(&server_addr, 0, sizeof(server_addr));
9      server_addr.sin_family = AF_INET;
10     server_addr.sin_addr.s_addr = htonl(INADDR_ANY);
11     server_addr.sin_port = htons(atoi(argv[1]));
12     if (bind(server_sock, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
13         perror("Bind failed");
14         return 1;
15     }
16     printf("UDP Echo Server đang chạy tại cổng %s...\n", argv[1]);
17     char buffer[BUFFER_SIZE];
18     struct sockaddr_in client_addr;
19     socklen_t client_len = sizeof(client_addr);
20     while (1) {
21         int bytes_received = recvfrom(server_sock, buffer, BUFFER_SIZE - 1, 0,
22 
```

udp_echo_client.c:

```

1  int main(int argc, char *argv[]) {
2      if (argc != 3) {
3          printf("Sử dụng: %s <IP Server> <Port>\n", argv[0]);
4          return 1;
5      }
6      int client_sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
7      struct sockaddr_in server_addr;
8      memset(&server_addr, 0, sizeof(server_addr));
9      server_addr.sin_family = AF_INET;
10     server_addr.sin_addr.s_addr = inet_addr(argv[1]);
11     server_addr.sin_port = htons(atoi(argv[2]));
12     char buffer[BUFFER_SIZE];
13     printf("Nhập tin nhắn (Ctrl+C để thoát):\n");
14 
```

Terminal Output:

Left terminal (Server):

```

nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang BTTL % ./udp_server 9000
UDP Echo Server đang chạy tại cổng 9000...
Nhập từ 127.0.0.1: Hello server
Nhập từ 127.0.0.1: tôi là Trang
Nhập từ 127.0.0.1: xin chào
Nhập từ 127.0.0.1: tôi là Trang
Nhập từ 127.0.0.1: 20235441
Nhập từ 127.0.0.1: đây là bài tập 3 trên lớp a
^C
nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang BTTL %

```

Right terminal (Client):

```

nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang BTTL % nc -u 127.0.0.1 9000
hello server
hello server
tôi là Trang
tôi là Trang
^C
nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang BTTL % ./udp_client 127.0.0.1 9000
Nhập tin nhắn (Ctrl+C để thoát):
> xin chào
Server echo: xin chào
> tôi là Trang
Server echo: tôi là Trang
> 20235441
Server echo: 20235441
> đây là bài tập 3 trên lớp a
Server echo: đây là bài tập 3 trên lớp a
> ^C
nguyenthutrang@MacBook-Pro-cua-NguyenThuTrang BTTL %

```

Giải thích:

Phía server chạy chương trình `udp_echo_server`, phía client chạy chương trình `udp_echo_client`:

- Server UDP nhận dữ liệu từ client mà không cần kết nối.
- Mỗi dữ liệu nhận được sẽ được gửi lại nguyên vẹn cho client.
- Client nhận và hiển thị lại dữ liệu phản hồi từ server.
- Chương trình hoạt động đúng theo mô hình **UDP Echo**.